

AI-POWERED SOFTWARE DEVELOPMENT TRACK • DIGILIANS

AI Spec-Driven Design

Hands-on Applied to AI Customer Support Assistant RAG System

Workshop 1 • Part 2

June 14, 2026 | 6:00 – 8:30 PM

What is Spec-Driven Design?

Definition

Spec-driven design is the practice of producing complete, machine-readable specification artifacts — requirements, architecture, API contracts, security controls, and test plans — before implementation begins.

The specifications become the single source of truth consumed by both AI coding assistants and human engineers.



Specs First, Code Second

No implementation until all artifacts are reviewed and approved



Living Documents

Artifacts evolve together — changing one triggers updates to all impacted docs



AI-Optimised Output

Documents are structured to be directly loaded by AI coding assistants as context

The Problem: Code-First Chaos

What happens without specs — the RAG project without this process



AI generates wrong architecture

Coding assistant picks Flask instead of FastAPI, REST instead of SSE, wrong DB schema — because nobody wrote the spec



Security added as afterthought

Prompt injection discovered in UAT. API keys hardcoded in frontend. No RBAC. Rewrite required.



Inconsistent component boundaries

Business logic leaks into routers. ChromaDB calls scatter across 8 files. Impossible to test.



Rework is expensive

Changing the LLM provider requires touching 12 files because LiteLLM wasn't planned for from the start



AI coding assistant goes rogue

Without referenced specs, Cursor fills in gaps with assumptions. Each session produces different patterns.



No test strategy = no confidence

"It works on my machine" — no golden dataset, no injection tests, no coverage metrics

Project Requirements: AI Knowledge Assistant

Upload PDFs → Semantic Search → Streaming Q&A → Source Citations · 60 minutes

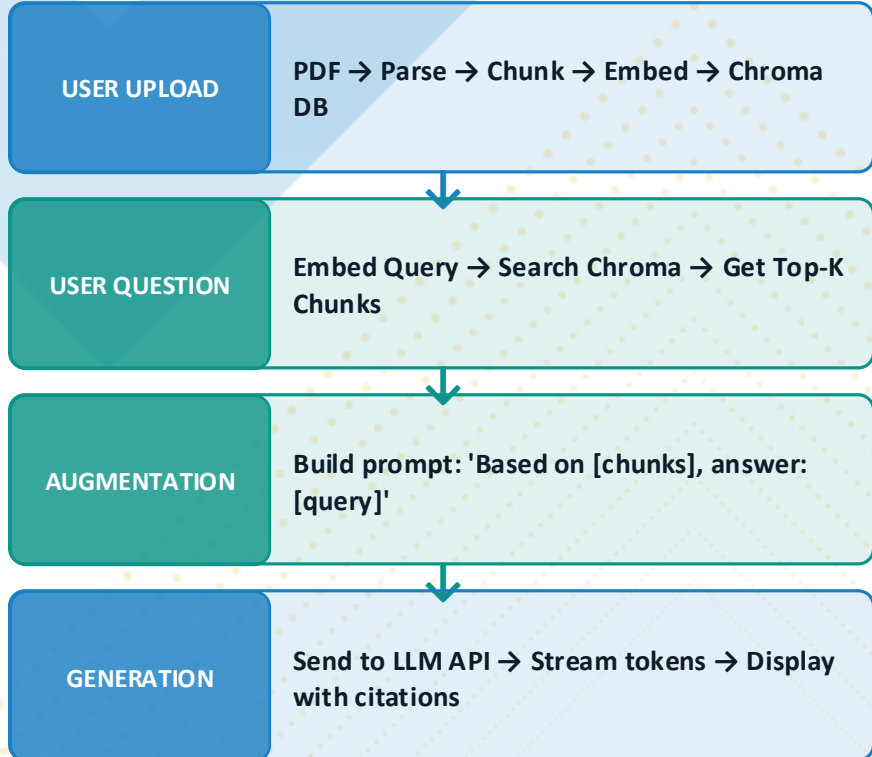
Tech Stack

Frontend	HTML Only (chat UI)
Backend	Python — FastAPI
Vector DB	Chroma (local)
LLM	Local LLM (Gemma4 2b) and Openrouter Free API
Embeddings	Text-embedding-3-small

Features

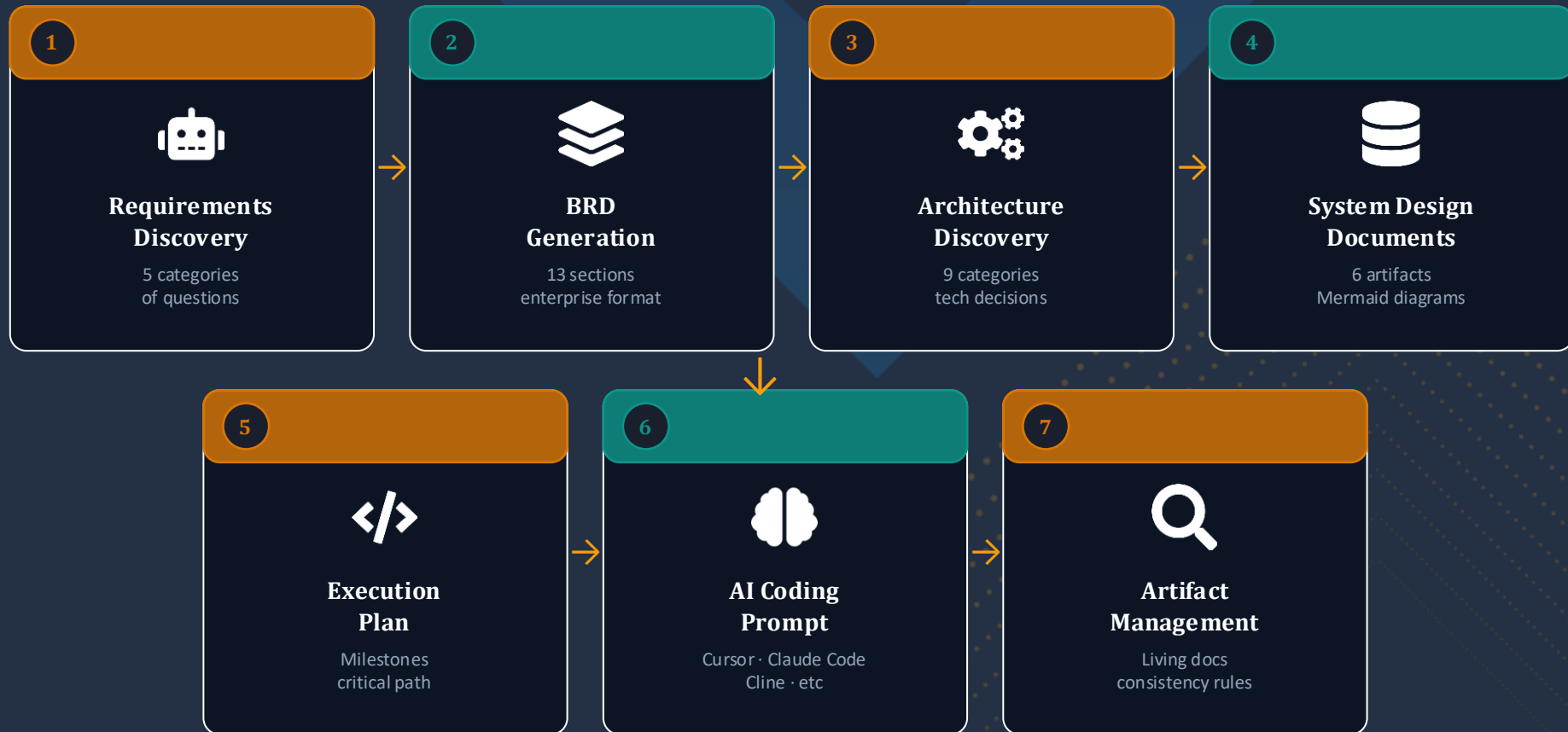
- Drag-and-drop PDF upload
- Semantic search across documents
- Streaming Q&A with real-time responses
- Source citations (document + page number)

System Architecture



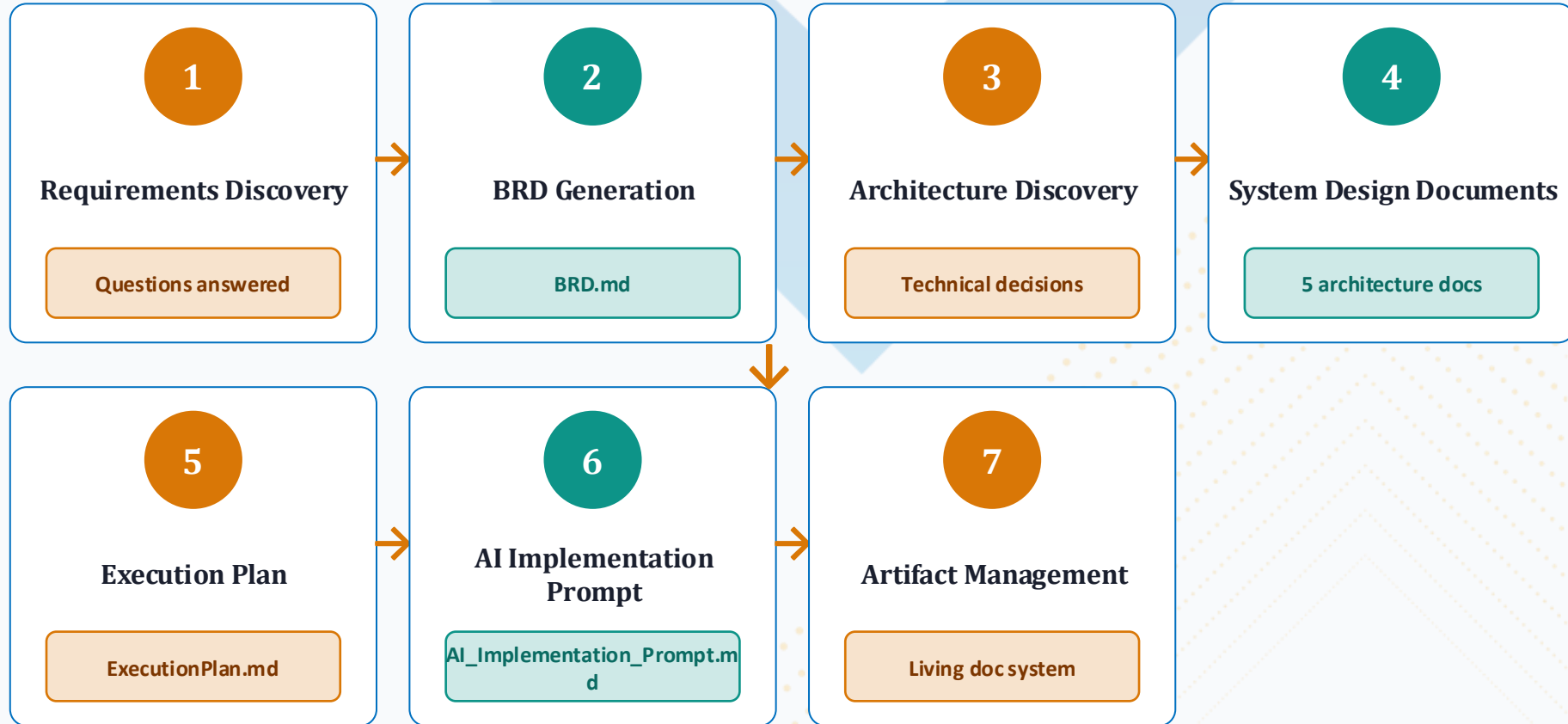
The Master Prompt That Generated All 9 Artifacts

7-phase role-stacked prompt · Senior BA + Architect + TPM + DevOps + Security + AI Lead



The 7-Phase Spec-Driven Process

Sequential — each phase gates the next



The 9 Specification Artifacts

BRD.md



Business objectives, scope, acceptance criteria, success metrics

SystemArchitecture.md



C4 diagrams, RAG flow, component boundaries, security arch

DatabaseDesign.md



SQLite schema, ChromaDB collection, Redis key schema

API_Specification.md



All endpoints, SSE schema, error codes, OpenAPI excerpt

SecurityDesign.md



STRIDE, injection patterns, RBAC, prompt boundaries

DevOpsDesign.md



Repo structure, Makefile, CI config, supervisord, backup

TestStrategy.md



Unit/integration/security tests, golden dataset, fixtures

ExecutionPlan.md



Epics, features, tasks, critical path, Gantt milestones

AI_Implementation_Prompt.md



Cursor/Claude Code prompt: rules, phases, validation gates

BRD.md — Business Requirements Document

BRD Section

What it defines for our RAG system

- | | |
|----------------------------------|--|
| ✓ Executive Summary | RAG system delivering cited answers, reducing ticket volume 40%, zero cloud cost |
| ✓ Business Objectives | 5 measurable objectives with metrics (BO-01 to BO-05) |
| ✓ Scope In / Out | Explicit in-scope list; Docker, mobile, SSO, fine-tuning all out-of-scope |
| ✓ Functional Requirements | 14 FRs with priority: FR-01 Chat UI → FR-14 Input sanitisation |
| ✓ Non-Functional Req. | 10 NFRs: ≤3s first token, ≤200ms cache hit, no PII to cloud, JSON logging |
| ✓ Acceptance Criteria | 6 AC: streaming answer, upload+index, cache hit, LLM switch, injection safe, /health green |
| ✓ Risks | 5 risks with likelihood/impact/mitigation: local latency, hallucination, injection |
| ✓ Success Metrics | ≥90% answer grounding, ≥30% cache hit, ≤3s TTFR, ≤30min setup time |

SystemArchitecture.md — Component Design

Ingestion Pipeline

- DocParser → Chunker
- EmbeddingService
- ChromaDB writer
- SQLite metadata write

Query Pipeline (RAG)

- InputSanitizer
- CacheService (Redis)
- VectorSearch (ChromaDB)
- PromptBuilder → LLMService

LLM Routing (LiteLLM)

- Primary: Gemma 4 2B (llama.cpp)
- Fallback: OpenRouter API
- Config: LLM_PROVIDER in .env
- Stream: token-by-token SSE

Data Flow — Query Path

User Query → Sanitize → Cache Check → Embed Query → Vector Search → Build Prompt → LLM Stream → Cite + Display

Key decision in the spec: ALL user input passes through InputSanitizer BEFORE any service call — enforced at the router boundary

SecurityDesign.md — Security-by-Design

Security is specified before coding — not discovered in production

STRIDE Analysis (RAG-specific)

Prompt Injection

High

InputSanitizer + prompt boundaries

Indirect Injection

High

Admin-only upload + content scan

API Key Leakage

Medium

Keys in .env; redacted from logs

Session Token Replay

Medium

24h TTL; Redis-backed

Unauthorised Doc Access

Medium

Auth + RBAC on all endpoints

Large File Upload DoS

Low

50MB limit; async queue

Prompt Injection Defence

SYSTEM INSTRUCTION

Use **ONLY** context. Say 'I don't know' if answer not in context.

== CONTEXT START ==

Retrieved chunks from ChromaDB (vetted documents)

== QUESTION START ==

Sanitised user query (injection patterns stripped)

⚠ The SecurityDesign.md prompt boundary template is directly copied into the code — no paraphrasing. Specs → code literally.

AI_Implementation_Prompt.md — The Coding Assistant Driver

Document Structure

Project Context
Stack, dependencies, deployment target

Referenced Documents
Load all 8 other artifacts before coding

Development Rules
10 non-negotiable rules (no hardcoded keys, etc.)

Architecture Constraints
Module boundaries from component diagram

Implementation Phases
5 phases with tasks and validation gates

Validation Gates
lint → type → unit → integration → security → coverage

Definition of Done
10-item checklist per feature before moving on

Works With



Cursor



Claude Code



Cline / Roo Code



Windsurf



GitHub Copilot Agent



OpenAI Codex

The AI prompt says: 'Read all artifacts first. Never guess requirements. Implement one phase at a time. Run validation gates before proceeding.'

Spec-Driven vs. Code-First

Applied to the AI Customer Support Assistant project

	Code-First	Spec-Driven
Architecture	LLM routing scattered across 12 files	All LLM calls in llm_service.py (spec mandates it)
Security	Injection discovered in UAT; full rewrite	STRIDE done first; sanitizer in code before demo
AI Coding Tool	Cursor guesses structure; inconsistent output	Claude Code reads 9 spec files; consistent structure
Testing	No tests until feature freeze; low confidence	Golden dataset + injection tests defined in TestStrategy
Team Alignment	Everyone has a different mental model	BRD = shared contract; disputes resolved by spec
Change Management	Change in LLM provider = 3-day refactor	LiteLLM abstraction spec'ed; LLM swap = 1 .env change
Time to Value	Fast start, slow finish (rework dominates)	Slow start, fast finish (code from spec is clean)

Spec-Driven + AI Coding Tools

How to load your artifacts into Cursor / Claude Code / Cline

1

Open project in your AI editor

All 9 artifact .md files live in /artifacts/ — they're already in the project

2

Load AI_Implementation_Prompt.md as system context

In Cursor: add to .cursorrules. In Claude Code: pass at session start. In Cline: add to .clinerules

3

Say: 'Implement Phase 1 per the execution plan'

The AI reads BRD + SystemArchitecture + DatabaseDesign and produces matching code

4

Run the validation gate before Phase 2

`make lint && make test-unit` — both must pass. The prompt tells the AI to wait for your confirmation.

5

Continue phase by phase — never skip gates

Security tests in Phase 5 catch injection vulnerabilities. Coverage gate ensures testability.

What You Can Do Now

The 9 spec artifacts for the AI Customer Support Assistant are complete and implementation-ready.



Open artifacts/ in Cursor or Claude Code — the specs are your codebase context



Run 'make setup && make run' — the Makefile and setup script are fully speced



Security controls are pre-coded in SecurityDesign.md — copy sanitizer.py verbatim



Test golden dataset is defined — implement tests before features (TestStrategy.md)



Execution Plan has your 5-week roadmap with critical path and Definition of Done



Apply this same 7-phase process to any future AI or software project

Artifact files in: /artifacts/ (9 files · all implementation-ready)